Borrowed Time project story

From a solo prototype to the submitted multiplayer game

This is the story of how I tested the idea, changed direction and improved the system. Read each diagram, then the two short paragraphs beneath it. Blue boxes mark additions or the focus of that stage. Milestones group related releases; they are not fourteen separate version numbers.

1 The first playable idea

Solo prototype before V1



Everything happens inside one browser at this stage.

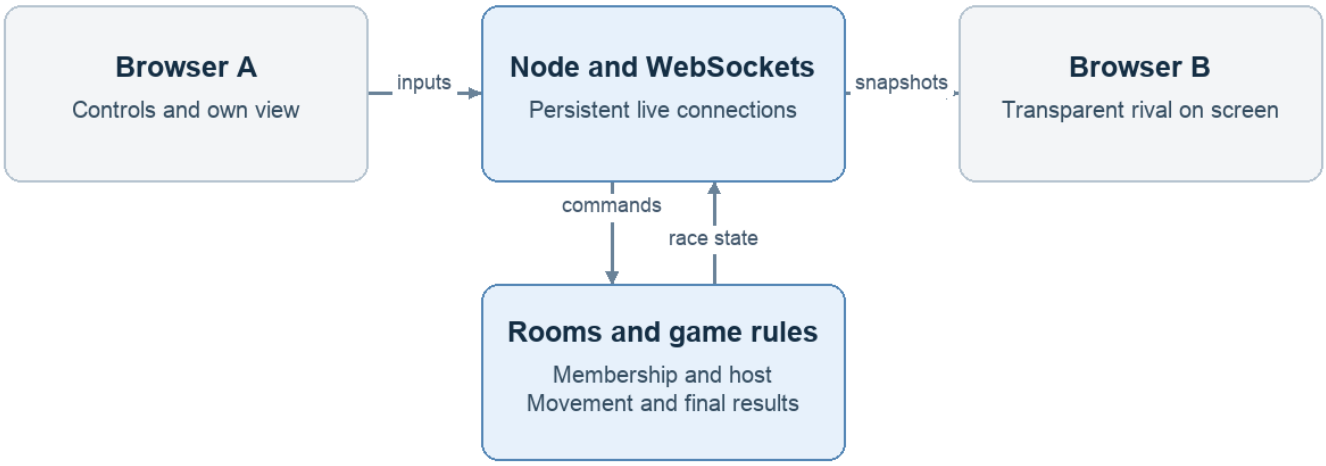
Assumptions and testing. I started with a small question: would a three-lane escape race feel fun if abilities cost the same time I needed to survive? The browser handled movement, obstacles, the countdown, Boost and Long jump. I played the basic loop and walked through the code before investing in artwork or infrastructure.

Findings and decisions. The idea was understandable enough to build on, but a solo run could not test the excitement of racing another person. I kept the rules separate from drawing and controls, then added a shared race server. This separation later let me change graphics without rewriting the whole game.

Evidence: Conversation decisions about the initial lane runner; docs/versions/V1_RELEASE.md; docs/PROJECT_GUIDE.md

2 A shared multiplayer race

V1



Both browsers send inputs and receive snapshots through their own live connection.

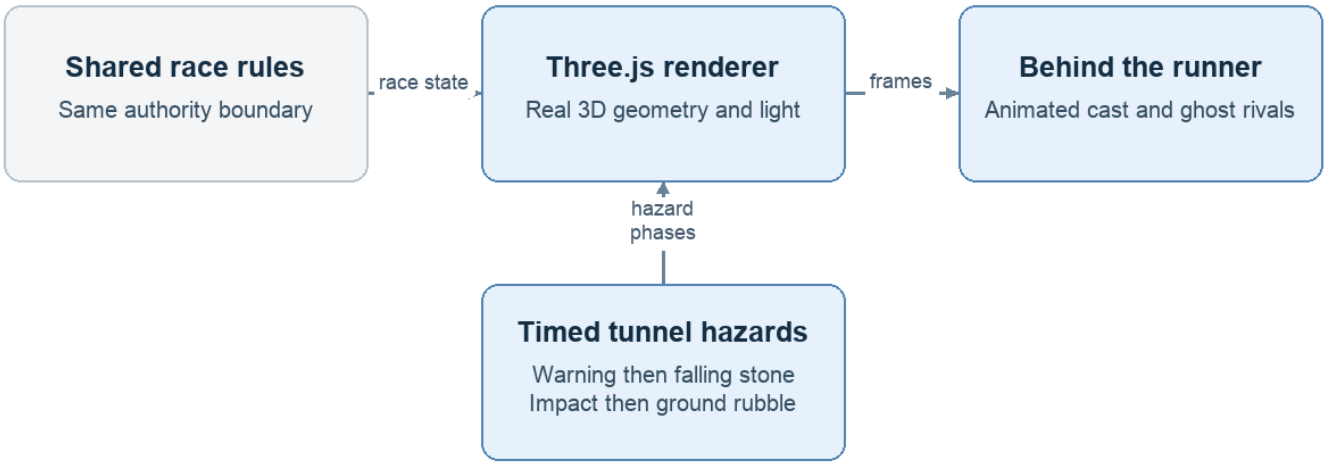
Assumptions and testing. I assumed private lobbies and visible opponents would create a stronger race feeling without needing accounts or matchmaking. V1 supported two to four people, a shared countdown, rematches and transparent rivals. Automated rule tests and real browser/socket races checked purchases, agreement on results, disconnects and host transfer.

Findings and decisions. The basic multiplayer flow worked: racers shared a course while keeping individual clocks and purchases. I made the server authoritative, meaning it decides movement, hits, time charges and winners; browsers display those decisions. Rooms stayed in one process in memory, which was simple for a competition but meant a restart would erase active lobbies.

Evidence: docs/versions/V1_RELEASE.md

3 Giving the race a world

V2 through V3 and V3 point 1



Three.js replaced simulated 2D depth; the browser still draws each frame.

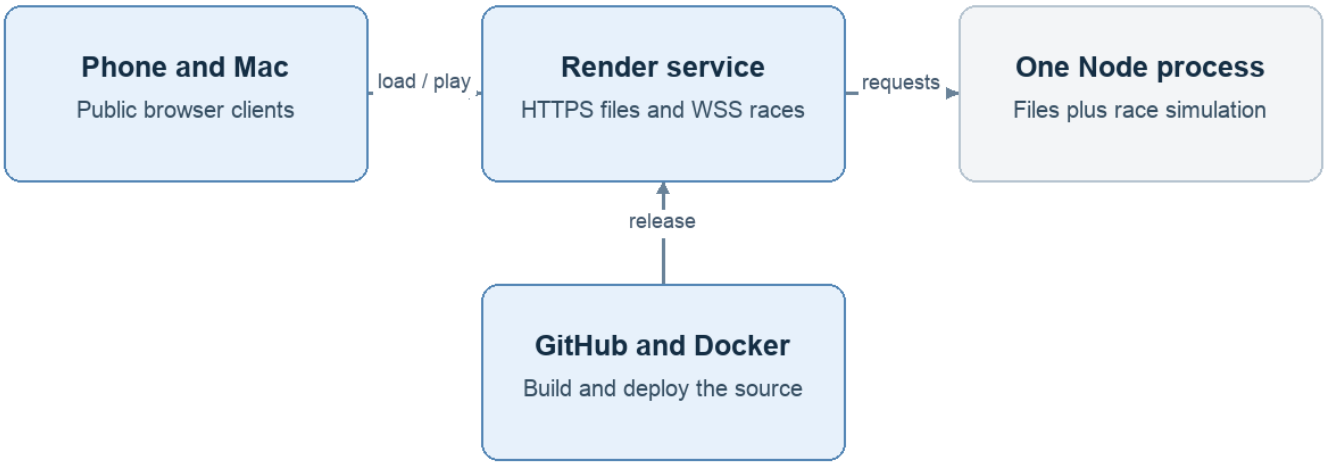
Assumptions and testing. I wanted the escape to feel like a place: a collapsing tunnel growing brighter toward its exit. Three.js replaced the Canvas 2D renderer, and the cast grew to four distinct animated characters. I tested shared debris timing, jumping clearance, independent ability effects and full multiplayer demos, alongside visual checks from the actual camera.

Findings and decisions. The new scene communicated the escape better, but early art still looked like a prototype. I refined materials, shadows and movement while preserving the backend. I also removed distance statistics from the HUD because they distracted from the clock. Rival visibility followed relative distance, and only the player buying Boost received their own boost presentation.

Evidence: docs/versions/V2_RELEASE.md; docs/versions/V3_RELEASE.md; docs/versions/V3_1_RELEASE.md; Conversation feedback on private boost effects

4 Putting it on the internet

V4



WSS is the encrypted WebSocket connection used over the public internet.

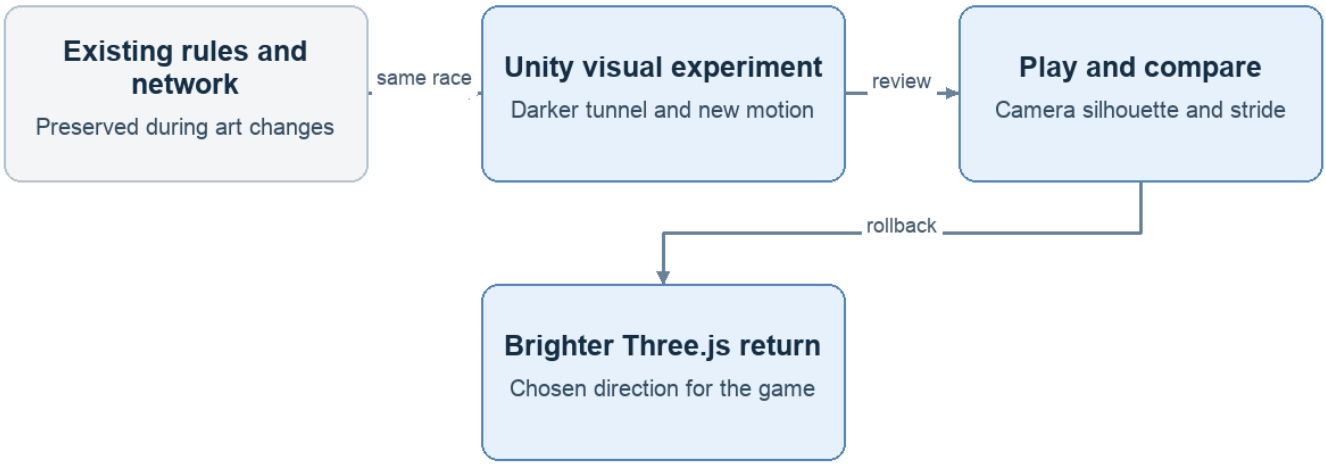
Assumptions and testing. Local browser tests were useful, but I needed to know whether a real phone and friends could play without the Mac hosting the game. I deployed the Node service to Render, using GitHub for source and Docker for packaging. Deployment checks loaded public assets and exercised two real WebSocket clients, countdowns, purchases and cleanup.

Findings and decisions. The game became shareable through a public link, allowing physical-phone feedback. I kept the free hosting plan. That introduced known limits: the race service can sleep while idle, and deployment or restart clears in-memory rooms. A working health endpoint was useful, but I retained a full race check because a healthy HTTP response alone does not prove multiplayer works.

Evidence: docs/versions/V4_RELEASE.md; docs/PROJECT_GUIDE.md

5 Testing a different visual direction

Unity study and V5 integration then V6 rollback



Blender and Unity explored art and animation; the submitted default returned to Three.js.

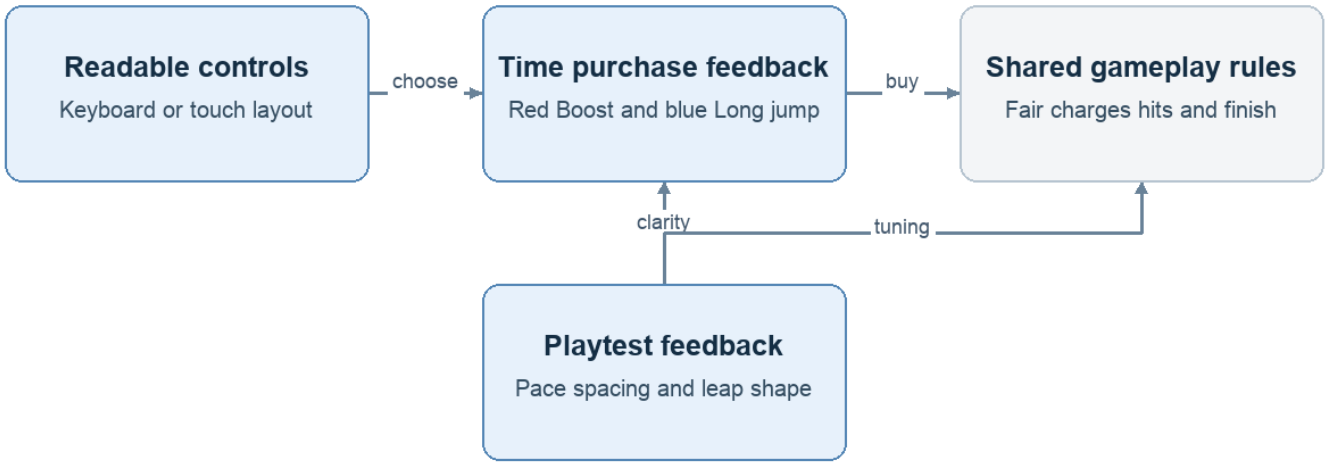
Assumptions and testing. I explored whether Blender assets and a Unity visual pass could deliver the atmosphere and animation I wanted. Separate previews tested proportions, running, jumping, landing and lane changes before Unity drew the actual game in V5. I compared camera distances and tried a fluid arm swing and stride with slight forward lean and no sideways banking.

Findings and decisions. A different engine did not automatically create the style I wanted. The Unity export also brought a larger download, about 38.2 MiB compressed in that checkpoint. After comparing the experience, I preferred the brighter pre-Unity game for a fun race. I restored Three.js as the default, preserved the experiment and backups, and carried the motion preferences into later polish.

Evidence: docs/versions/V5_RELEASE.md; docs/versions/V6_RELEASE.md; Conversation visual direction and rollback decisions

6 Making the choices visible and useful

V6 tuning and later forward leap refinement



The accepted ability prices became Boost 5 seconds and Long jump 4 seconds.

Assumptions and testing. Players enjoyed racing but missed the abilities, did not always recognize time as currency, and found survival too easy. I tested faster running, harder routes, clearer spending feedback and contextual button hints. Phone play also exposed a leaderboard blocking controls. Later, I compared a high-looking paid jump with a lower leap that carried the racer forward.

Findings and decisions. I moved the HUD and controls into the play area, used stronger red and blue ability colors, and adapted the layout to touch or keyboard. Boost dropped from six to five seconds after feedback; obstacle sets spread out while base speed increased. The four-second Long jump became a forward burst and a credible alternative to Boost. Time Shield remained a discussed idea, not a submitted feature.

Evidence: docs/versions/V6_MOBILE_HUD.md; docs/versions/V6_PACING_UPDATE.md; docs/versions/V6_RELEASE.md; docs/versions/V8_8_FORWARD_LEAP.md

7 Six maps with one consistent game

V7 and V8 through the V8 point 12 detail pass



Tunnel, ship, volcanic jungle, frozen summit, orbital station and sandstorm canyon.

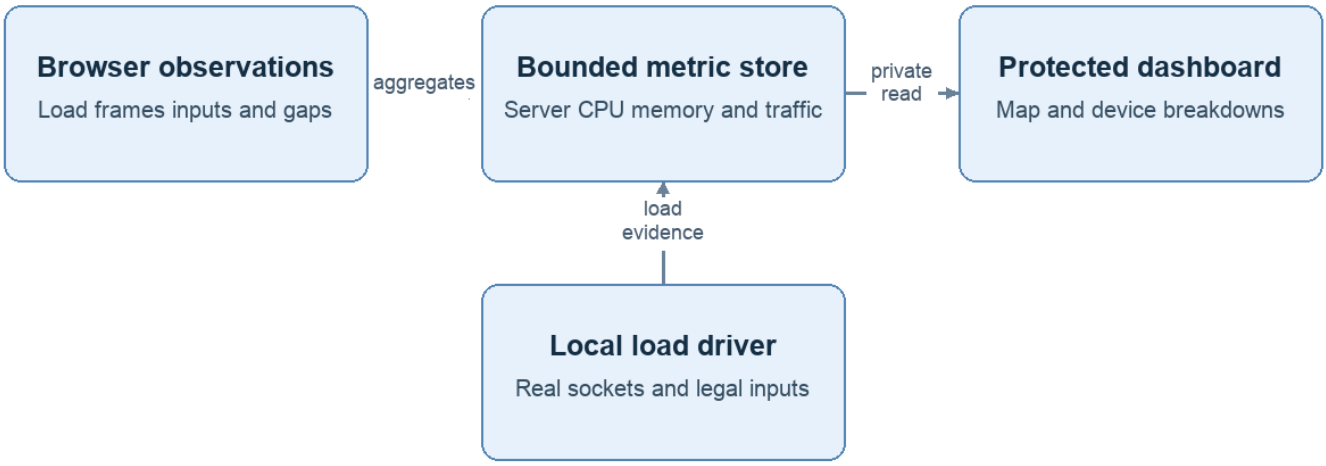
Assumptions and testing. I assumed variety would encourage another run if every map kept familiar mechanics. The tunnel grew into six settings with 24 cosmetic characters, map thumbnails and voting. Solo gained computer racers; optional character choices and session wins added identity. I checked all-map rules and multiplayer flows, then reviewed obstacle cues, starts, finishes, music and detailed models.

Findings and decisions. Reusing collision categories let cannonballs, hot rock, ice and station fragments feel themed without creating six separate rule engines. I kept a common behind-runner camera and ability prices. Contrast markers made hazards easier to read; clearer debits showed time spending. I chose steady map music, preserved artist credits and refined the brighter materials. More characters and detail also increased the loading cost I needed to measure next.

Evidence: docs/versions/V7_THREE_MAPS.md; docs/versions/V8_SIX_MAPS.md; docs/versions/V8_1_READABILITY_FEEDBACK.md; docs/versions/V8_2_CLOCK_START_FINISH.md; docs/versions/V8_6_SOLO_RIVALS.md; docs/versions/V8_7_CHARACTERS_SESSION.md; docs/versions/V8_9_RECORDED_MUSIC.md; docs/versions/V8_12_FULL_DETAIL_PASS.md

8 Measuring before claiming scale

Capacity baseline and protected monitoring



Operational data requires a private key; the game does not receive that key.

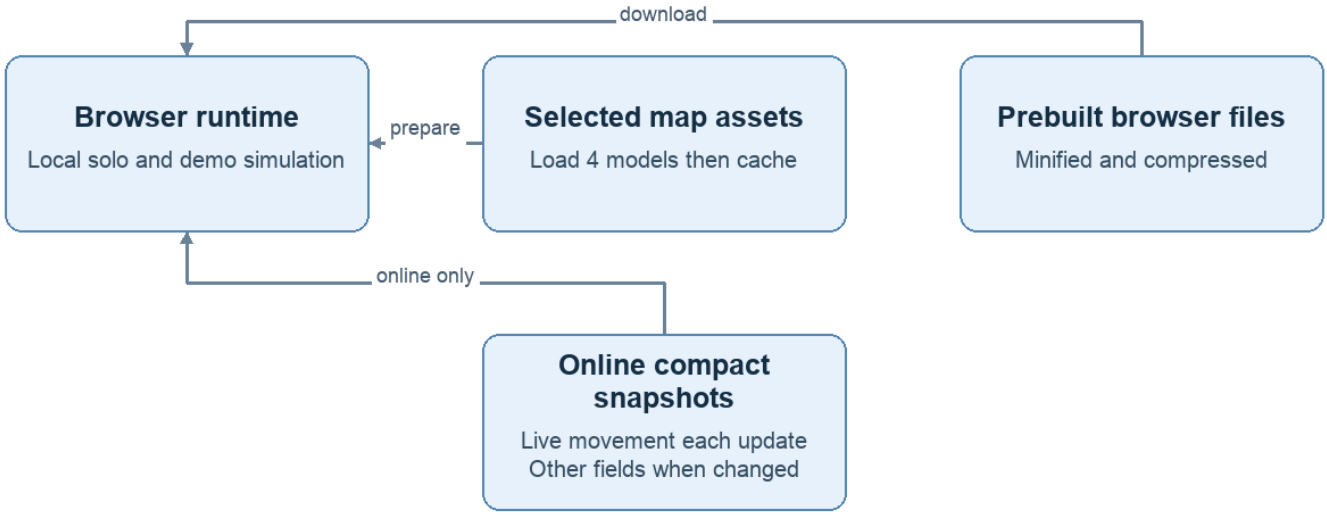
Assumptions and testing. A possible burst of visitors raised a different question: could the service cope, and how would I spot a bottleneck? I ran bounded local socket tests and added a protected dashboard for occupancy, CPU, memory, bandwidth, loading, frames and network timing. Reports separated maps, estimated mobile/desktop devices and play modes; retention grew to 30 minutes so a full map test stayed visible.

Findings and decisions. The local baseline kept roughly 20 updates per second at 100 connected racers, but this did not prove capacity on Render Free. It exposed clearer waste: each old demo consumed four sockets and a room, all 24 models preloaded, and unchanged data repeated in snapshots. I prioritized those costs. Monitoring remained bounded, aggregate and in memory; restarts erase history unless I export the evidence.

Evidence: docs/CAPACITY_BASELINE.md; docs/MONITORING.md

9 Removing avoidable downloads and race traffic

Browser demo selected loading cache and compact updates



MP3 music already used audio compression; only the chosen track loads when needed.

Assumptions and testing. I expected fewer unnecessary bytes and less demo work to help loading and server headroom without changing gameplay. I moved the demo into the browser, loaded only the selected map cast, added content-version caching, and bundled/compressed browser code at build time. Multiplayer snapshots stopped repeating unchanged roster, standings and hazard information. I measured before and after, then used phone tests to check the experience.

Findings and decisions. A short local demo comparison went from four race sockets to zero. Initial tunnel model transfer fell about 83%, from 13.24 MB to 2.23 MB; the compact protocol reduced local test traffic about 59%, without proving lower CPU. One first Orbital load felt slow, while cached retests were quick, so I did not call the cold-load issue disproved. Fewer downloads helped efficiency, but internet multiplayer still needed separate testing.

Evidence: docs/LOCAL_DEMO.md; docs/MAP_LOADING_CACHE.md; docs/LOADING_BANDWIDTH.md

10 Separating file delivery from live races

Public static site and race service split



The race process no longer serves game asset downloads.

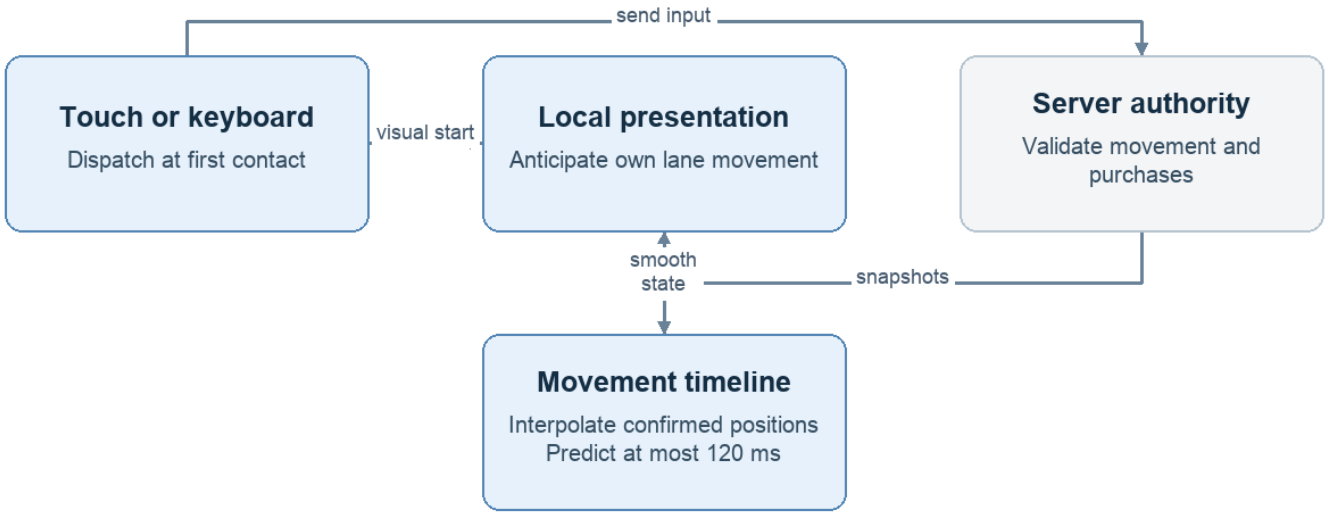
Assumptions and testing. I suspected that serving large files and simulating races in one process could create avoidable competition for resources. I separated a static frontend from the Node race service, first locally and then publicly. I also measured when updates were scheduled, queued by the application and received by the browser, rather than treating every late arrival as slow game logic.

Findings and decisions. Public checks confirmed that the race origin served no game asset bodies after the split. A phone sample showed fewer holds, and I experienced smoother play, but prediction and other conditions also differed, so I could not credit the split alone. Local profiling did not reproduce the larger hosted pauses. I kept the split and investigated both application timing and delivery after sending, without declaring a proven host or Wi-Fi fault.

Evidence: docs/SPLIT_DELIVERY.md; docs/SERVER_PAUSE_PROFILE.md; docs/NEXT_SESSION.md public split delivery record

11 Making delayed updates less visible

Control timing anticipation and bounded recovery



Prediction changes the displayed motion; it does not award extra game distance or time.

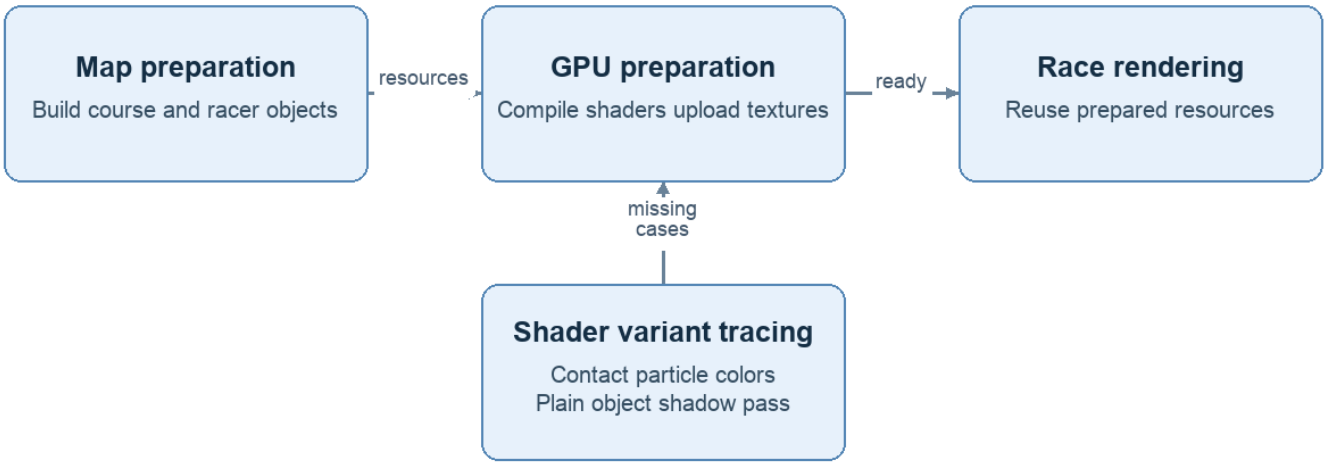
Assumptions and testing. I noticed brief freezes followed by catch-up, especially early in public races. I instrumented touch dispatch, acknowledgment and rendered response, then added early touch handling and immediate visual lane anticipation. I tested interpolation, short forward prediction and bounded recovery from delayed snapshots using deterministic replays and phone-plus-Mac races.

Findings and decisions. Some local tests looked better, but public skips persisted. That showed why a Mac-hosted race could not establish Render-to-phone behavior. I kept server decisions authoritative and capped forward prediction at 120 ms so missing updates could not produce unlimited invented motion. Recovery eased back toward confirmed history rather than jumping abruptly. These changes addressed presentation, not the reason an update arrived late.

Evidence: docs/FORWARD_RECOVERY.md; docs/superpowers/plans/2026-10-28-bounded-prediction.md; docs/NEXT_SESSION.md control timing and lane anticipation records

12 Preparing graphics before the start

Shader warmup object preparation and late variant fixes



A shader is a small GPU program used to draw a surface, effect or shadow.

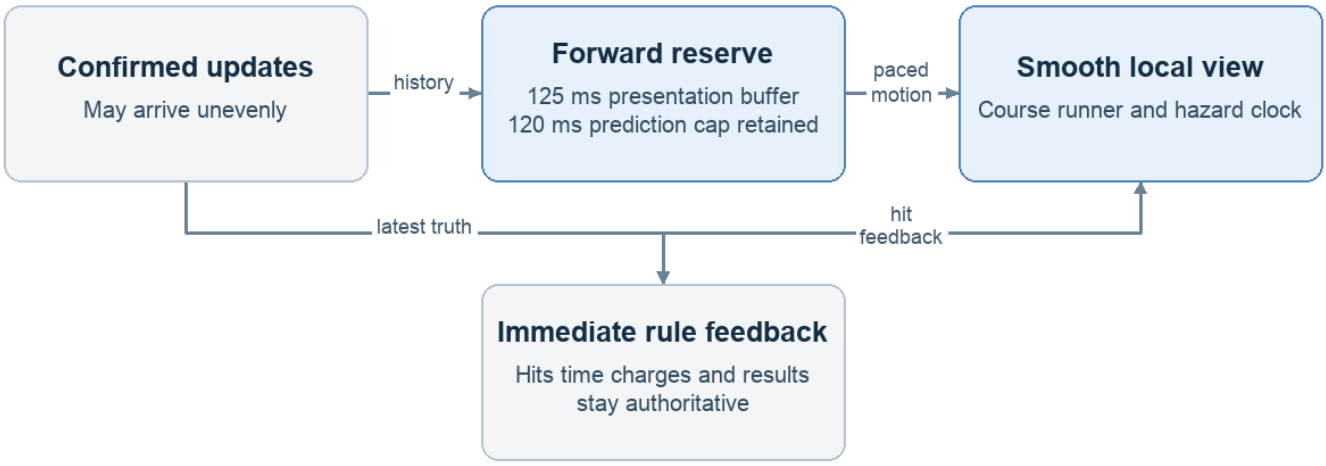
Assumptions and testing. My experience suggested first-use graphics work could feel like the skips, even when CPU metrics looked normal. I tested that hypothesis. Tunnel preparation compiled representative materials, initialized textures and drew hidden passes before play. I then prepared course objects and racers ahead of time, counted new allocations/programs and traced the two shader variants still appearing during races.

Findings and decisions. The remaining variants came from particle instance colors and an untextured shadow caster. Preparing those exact layouts removed new shader programs in the measured Tunnel retest; tracked race object creation was also zero. Yet update gaps and holds remained. I kept the graphics improvements because they removed real first-use work, while separating that finding from the unsupported claim that shaders explained every skip.

Evidence: docs/RACE_OBJECT_PREPARATION.md; docs/TUNNEL_SHADER_VARIANTS.md; docs/NEXT_SESSION.md public shader fix retest

13 Finding the cause of visible catch up

Wi Fi comparison forward reserve and collision smoothing



The reserve absorbs short delivery variation; it trades some forward visual latency for smoothness.

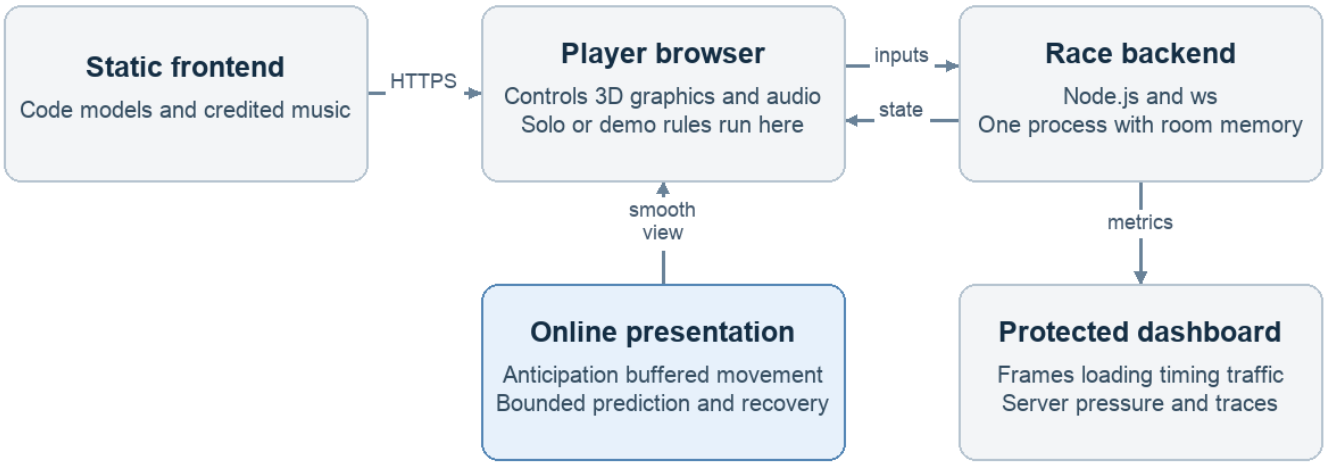
Assumptions and testing. To distinguish connection effects, I compared the same public build on phone Wi-Fi and cellular. I then added a 125 ms reserve of confirmed forward movement, leaving the 120 ms prediction cap unchanged. When catch-up remained, a small diagnostic trace captured motion and collision context around the event. I compared those traces with the eye test before changing anything else.

Findings and decisions. One cellular trial had no arrival gaps above 100 ms, while the earlier Wi-Fi trial had ten; that suggested connection-path variation, not proof of a faulty router. The reserve reduced reported holds from eight to three in another comparison, but some jumps remained. Traces identified four collision-driven visual reanchors. I smoothed those corrections while keeping hits and time penalties immediate; I felt smoother collisions.

Evidence: docs/OWN_FORWARD_BUFFER.md; docs/CATCHUP_TRACE.md; docs/NEXT_SESSION.md cellular comparison and collision smoothing records

14 The version I submitted

Accepted runtime 86362c2 and submission on October 29 2026



Online rules run on the server. Solo and demo reuse the same rules in the browser.

Assumptions and testing. I treated readiness as an end-to-end check, not a claim of perfect latency. Shared presentation changes applied to all six maps; automated network checks exercised voting, purchases, movement and cleanup, and I played every map on the public build. I reviewed the live game URL, a real 3:2 Tunnel screenshot, the description and learning/next-step text before I submitted the form myself.

Findings and decisions. I accepted all six maps as feeling good overall, submitted the project, and shared it on LinkedIn and Twitter. The final product kept six maps, solo CPU rivals, two-to-four-player online races, cosmetic roles and session wins. The final lobby flow is host Start match, seven seconds of voting, then a three-second race countdown. I retained free hosting and a single in-memory race process; broad device coverage, guaranteed peak capacity and zero lag remain unproven.

Evidence: docs/SUBMISSION_READINESS.md; docs/MAP_VOTING_FLOW.md; docs/NEXT_SESSION.md accepted six-map baseline; Conversation final form review and owner submission confirmation